

INTERNAL TECHNICAL & BUSINESS REFERENCE — ANALYSIS ONLY

Construction Automation — Complete Field Guide

Everything verified directly from the codebase at `construction-extraction-test`:
business flow, architecture, database, authorization, extraction pipeline, and a ready demo
sequence — written for a team walkthrough, not just for engineers.

Next.js 16 · React 19

Supabase / PostgreSQL

Gemini Vision OCR

AssemblyAI STT

Part 1

Project Overview

1–4. What it is, the problem, the users, why a company needs it

This is a **construction progress tracking and approval system**. Real construction projects are broken into *work items* (e.g. "Pour concrete foundation — Building A, 500 m³"). Workers physically do that work every day, and someone has to record how much got done, someone has to check that record is accurate, and someone has to approve it before it counts toward the project's official progress and billing. This app digitizes that entire chain: from a worker typing/photographing/speaking an update in the field, to a Contractor approving it, to a dashboard showing the project's real progress.

5–6. The old manual process, and what's automated

The manual version of this is a paper daily-log or a WhatsApp message ("poured 40m³ today") that a supervisor later re-types into an Excel sheet, which someone else re-checks against site photos, then a project manager manually recalculates percentage-complete for a report. Each hand-off is slow and error-prone. This app automates: capturing the update (text/photo/voice), reading structured data out of it (OCR/speech-to-text + parsing), routing it through a fixed two-step review chain, and recalculating cumulative progress automatically the moment something is approved — no manual re-typing or recalculation anywhere in that chain.

7–8. From login to an approved number on a dashboard — a concrete example

Worker reports: "Completed 40 cubic meters of concrete."

1. Worker types/photographs/speaks this on their `/workflow/worker` dashboard.
2. The app parses it into structured data — quantity 40, unit m3 — and inserts a row into `extraction_submissions` (status SUBMITTED). Nothing is "official" yet.
3. Their Subcontractor (Foreman) sees it in their review queue, optionally corrects the number, and **forwards** it — this writes a `user_validations` row. Still not official; this only moves it to the Contractor's queue.
4. The Contractor (Supervisor) **approves** it. Only now does the app write a row into `unified_records` (status: APPROVED, accepted_quantity: 40) — this is the moment the number becomes real.
5. Every dashboard reading "cumulative progress" for that work item re-sums every APPROVED `unified_records` row for it, live, on every page load — so the Contractor's dashboard, the project's overall progress ring, and the department breakdown all update immediately.

Note the actual real flow differs slightly from a naive "worker → contractor → dashboard" picture: there are always **two separate gates** — a Subcontractor forward and a Contractor approval — before anything counts. This is verified directly in `lib/workflow.ts` and covered fully in Part 13.

Part 2

Technology Stack

Read directly from `package.json` — nothing here is assumed.

LAYER	TECHNOLOGY	NOTES
Framework	Next.js 16.3.0 (App Router, Turbopack)	Server Components by default; <code>proxy.ts</code> replaces the old <code>middleware.ts</code> convention in this Next version.
UI library	React 19.2.8	With Server Actions ("use server" functions called directly from forms).
Language	TypeScript 5	Whole codebase is <code>.ts</code> / <code>.tsx</code> .
Styling	Tailwind CSS 4	Utility classes + a small CSS-variable design-token system in <code>app/globals.css</code> (Steel Blue / Teal / Green palette).
Icons	lucide-react	Only icon library used app-wide.
Auth & DB client	@supabase/supabase-js, @supabase/ssr	Real Supabase Auth (email/password); server + service-role clients.
Database	Supabase (PostgreSQL)	18 tracked migrations, plus several core tables created directly in the Supabase dashboard (Part 8).
AI / OCR	@google/genai (Gemini)	Used only for handwriting/photo OCR — <code>lib/ocr.ts</code> , model default <code>gemini-3.6-flash</code> .
Speech-to-text	AssemblyAI	Called via raw <code>fetch</code> REST calls, no SDK dependency — <code>lib/stt.ts</code> .
Spreadsheet parsing	xlsx (SheetJS)	Excel/CSV project import.
File storage	Supabase Storage	Private buckets: handwritten, voice, live-updates.
Testing	<i>none installed</i>	No Jest/Vitest/Playwright in <code>package.json</code> — verification is <code>tsc</code> + <code>eslint</code> + <code>next build</code> + manual browser testing.

Did we use Python anywhere? — No. Confirmed: zero `.py` files anywhere in the repository, and no `child_process/spawn/Python` references in any source file. Every piece of "smart" processing (OCR, speech-to-text, text parsing) runs either as a hosted API call (Gemini, AssemblyAI) from Node.js/TypeScript server code, or as plain TypeScript regex-based parsing (`lib/progressParsing.ts` , `lib/construction.ts`). There is no separate Python microservice, script, or notebook anywhere in this project.

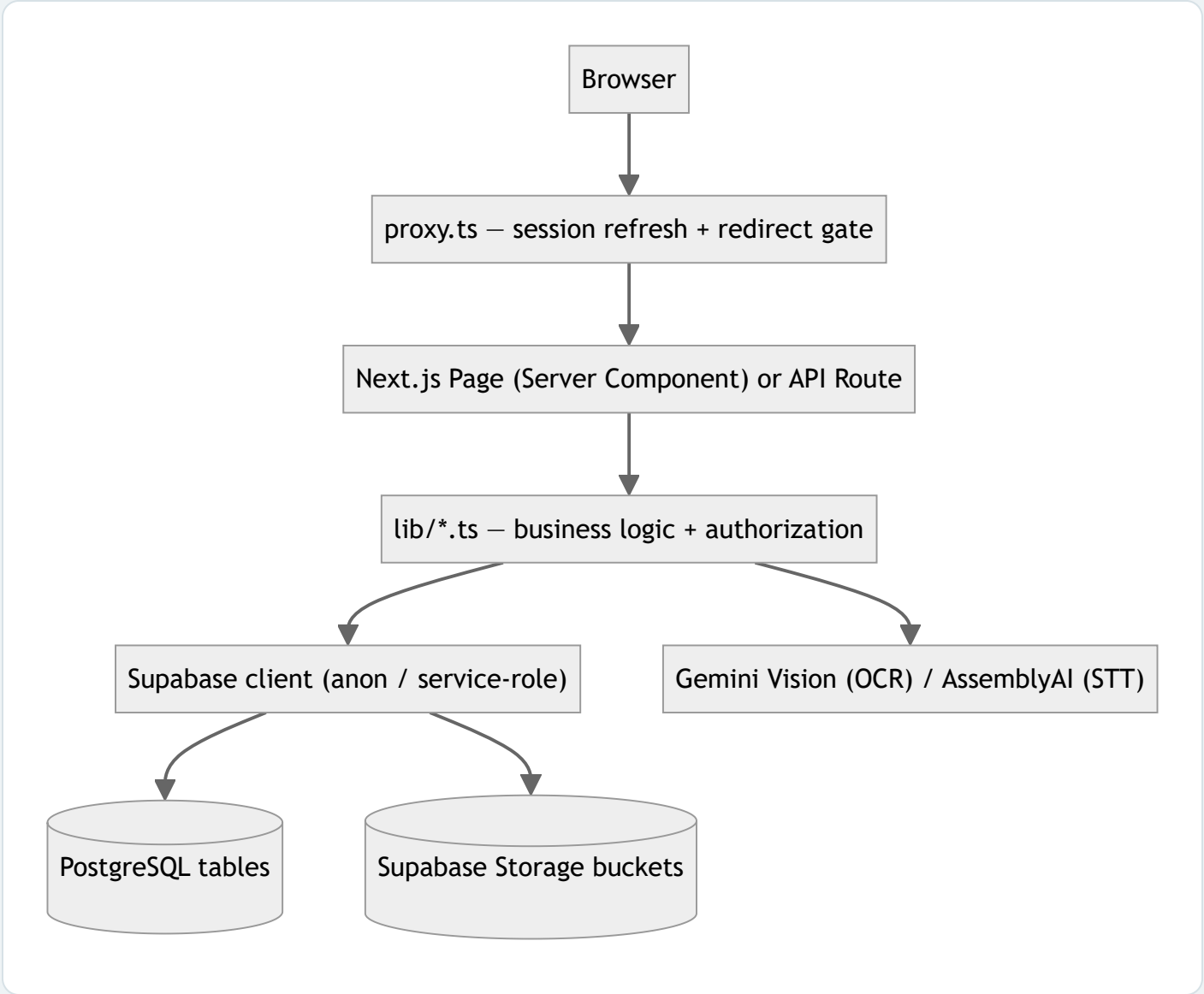
Part 3

Complete Folder / Code Architecture

FOLDER	WHAT LIVES THERE	KEY FILES
app/	Every page and API route (Next.js App Router — folder path = URL path). Each role has its own subfolder under app/workflow/ .	app/login/page.tsx , app/workflow/worker/page.tsx , app/admin/setup/page.tsx , app/api/**/route.ts
components/	React UI — organized by area: components/auth/ (login), components/workflow/ (all four dashboards + shared shell), components/admin/ (Admin Setup tabs), components/ui/ (Card/Button/Badge/Input primitives).	WorkerTabs.tsx , ForemanTabs.tsx , ContractorTabs.tsx , AdminSetupPanel.tsx , DashboardShell.tsx
lib/	All business logic and database access — pages/components never talk to Supabase directly for anything non-trivial; they call a function here.	workflow.ts (submission/review/approval), dashboard.ts , admin.ts , delegation.ts , authContext.ts , session.ts , chat.ts , liveUpdates.ts , ocr.ts , stt.ts , calculations.ts , progressParsing.ts
supabase/migrations/	18 SQL files, applied in order, that create/alter tables, indexes, and storage buckets. Not the whole schema — see Part 8.	00000000000001_schema.sql ... 00000000000018_chat_direct_messages.sql

FOLDER	WHAT LIVES THERE	KEY FILES
public/	Static assets only — logo, login hero photo, previously-used reference screenshots. Never user-uploaded content.	agentic-atoms-logo.png , living-blueprint.jpg
proxy.ts (repo root)	Next.js 16's renamed middleware.ts — the first checkpoint on every request to a protected path.	—

The layered architecture



Reading the arrows: the browser never talks to Supabase or Gemini/AssemblyAI directly — every request goes through a Next.js page or API route first. That route calls into `lib/`, which is where every permission check happens (never trusting anything the browser claims about who it is or what role it has — see Part 17). Only `lib/` code holds a Supabase client; two different clients exist — a normal one (respects Row Level Security) and a *service-role* one (bypasses RLS entirely, used only for tables that are deliberately locked to "server code only," like chat and live updates).

Part 4

Authentication / Login

Authentication vs. Authorization — the beginner version:

Authentication answers "who are you?" — checking an email+password against Supabase Auth.

Authorization answers "what are you allowed to do, now that we know who you are?" — e.g. a Worker is authenticated the same way an Admin is, but only the Admin is *authorized* to create a project. This app checks both, separately, on every request: `lib/session.ts` handles authentication, `lib/authContext.ts` / `lib/workflow.ts` / `lib/delegation.ts` handle authorization.

How it actually works

1. **Real Supabase Auth login** (`app/login/actions.ts` , the `login` Server Action): calls `supabase.auth.signInWithPassword({ email, password })`. If the account isn't linked to a `public.users` row, it immediately signs the session back out rather than leaving an "authenticated but unknown" state.
2. **Demo login** (`lib/demoAuth.ts` , `app/login/demoActions.ts`): *not* a bypass. It's gated by a server-only `DEMO_MODE=true` env var; the browser only ever sends one of 11 fixed role keys (never an email); the server resolves that key to a pre-provisioned real account's email, confirms it's already linked to Supabase Auth, then mints a genuine session via a magic-link mechanism (`auth.admin.generateLink + verifyOtp`) — the same technique used for password-reset links. From that point on, a demo session is indistinguishable from a normal login.
3. **Session identification** (`lib/session.ts` , `getCurrentUser()`): calls `supabase.auth.getUser()` — deliberately *not* `getSession()`, because `getUser()` re-verifies the token against Supabase's server every time, while `getSession()` just trusts a cookie. It then looks up the matching `public.users` row by `auth_user_id`.
4. **Role identification** (`lib/authContext.ts`): `isAdminUser()` checks `users.user_role = 'ADMIN'` directly; every other role (WORKER/FOREMAN/SUPERVISOR/MANAGER) comes from that user's single Active row in `user_project_roles`, which also carries their project and department.
5. **proxy.ts** (Next 16's `middleware.ts`): runs before the page even starts rendering, on `/workflow/**`, `/admin/setup/**`, and `/account`. It refreshes the Supabase session cookie and — if there's no session at all — redirects to `/login?next=<original path>`.

This check is explicitly documented in the code as "optimistic": the real, authoritative check happens again inside every page via `requireCurrentUser()`.

6. **Unauthorized users:** redirected to `/login` by `proxy.ts` (no session) or, inside a page, redirected back to `/workflow` if their role doesn't match that page (e.g. a Worker hitting `/workflow/supervisor`).

7. **After successful login:** redirected to `?next=` if present, otherwise `/workflow` — a small router page that checks the user's real role and sends them to the correct dashboard.

Which role goes where

ROLE (DB)	UI LABEL	DESTINATION
ADMIN	Admin	<code>/admin/setup</code>
WORKER	Worker	<code>/workflow/worker</code>
FOREMAN	Subcontractor	<code>/workflow/foreman</code>
SUPERVISOR / MANAGER	Contractor	<code>/workflow/supervisor</code>

Part 5

All User Roles

The database only defines four role codes: ADMIN, SUPERVISOR/MANAGER (treated identically everywhere — "the same Contractor-level role," per the code's own comment), FOREMAN, WORKER. "Electrical Foreman," "HVAC Worker," etc. are **not separate roles** — they're a role + a department (Electrical, Plumbing, Concrete, HVAC...) combination. The demo login's 11 accounts are demo *data*, one per role+department combination — not additional application roles.

ROLE	PURPOSE	MAIN SCREENS	CAN DO
ADMIN	Owens the whole system — company/project/department setup, user accounts, delegation.	/admin/setup	Everything: c companies/p items/users, approve/edit anyone.
SUPERVISOR / MANAGER ("Contractor")	Reviews and approves progress; owns overall project value/status.	/workflow/supervisor , /workflow/supervisor/history	Approve/edit their own dep dashboard/Li Updates/Con delegation, a
FOREMAN ("Subcontractor")	Manages a department's workers and forwards their submissions for Contractor approval.	/workflow/foreman	Assign/remov in their own c forward/corre Updates/Con
WORKER	Does the physical work and reports progress on it.	/workflow/worker	Submit progr only against v them; view th work; use Co

Parts 6 & 21

Login Page

Files: `app/login/page.tsx` , `components/auth/LoginBlueprintVisual.tsx` ,
`components/auth/LoginForm.tsx` , `components/auth/DemoLoginPanel.tsx` .

Layout — one fixed screen, two regions

The page is a single `<main>` sized to exactly fill the viewport below the top nav (`flex-1 min-h-0 overflow-hidden`) — no page scrolling, no second background layer. It's split left/right, not stacked: a "Living Blueprint" photo panel (~58% width on desktop, a compact banner on mobile) and the sign-in panel (~42%) sit edge-to-edge with no border/shadow/card between them, so it reads as one continuous surface, not two screens.

ELEMENT	WHAT / WHY	UI-ONLY OR BACKEND-CONNECTED
Left visual (<code>public/living-blueprint.jpg</code>)	The actual reference photo (a miniature building model, crane, greenery, blueprint paper), with a thin Steel-Blue→Teal gradient over it for text legibility. Purely atmospheric branding.	UI-only, decorative.
Agentic Atoms logo + name	Brand identity, shown on both panels at every screen size.	UI-only.
"Welcome Back" heading + subtext	Orients the user.	UI-only.
Demo Login (Role → Demo User dropdowns + Enter Demo)	Compact 2-dropdown demo-account picker, dev/demo environments only.	Backend: submits to the real <code>demoLogin</code> Server Action (Part 4).
Email / Password fields + Sign In	Standard credential login.	Backend: the real <code>login</code> Server Action → <code>supabase.auth.signInWithPassword</code> .
Forgot your password?	Link to <code>/reset-password</code> .	Backend: <code>requestPasswordReset</code> Server Action.
Error messages	Shown for an expired/invalid email link (<code>?error=</code>) or a failed sign-in.	Backend-driven text, UI-only rendering.

Confirmed: the page is fixed-viewport by design (no intentional scroll), the image lives only inside the left panel (never behind the whole page), and there is no full-screen overlay anywhere — the earlier `BackgroundFX` animated-particle component was removed from the app entirely (Part 20).

Part 7

Admin Flow — Setup & Excel Import

Login (Admin) → /admin/setup . components/admin/AdminSetupPanel.tsx renders 7 tabs; every tab's server-side data is pre-filtered to the caller's actual scope before it ever reaches the browser (so a delegated Contractor never even receives another project's data in the page payload, not just a hidden button).

TAB	GATE	WHAT IT DOES
Companies	Real Admin only	Create/view companies — the root of Company → Project → Department → Work Item.
Projects	PROJECT_MANAGEMENT	Real Admin: full "New Project" form + edit any project. Delegate: edit only their delegated projects, no creation form at all. Each project row embeds the Excel Import button.
Departments	DEPARTMENT_MANAGEMENT	Create a department (optionally pre-filled from a "standard department" list — Electrical, Plumbing, Concrete, HVAC...) or edit an existing one.
Work Items	WORK_ITEM_MANAGEMENT	Create/edit work items: code, description, planned quantity + unit, scheduled value, retainage %, dependencies on other work items.
Users	USER_MANAGEMENT	Create users and assign role/department; "Link Login" issues a one-time password and links a real Supabase Auth account. Admin-role creation is always blocked for a delegate.
Delegations	Real Admin only	Grant/revoke delegated access (Part 18).
Communication	Anyone who can reach Admin Setup	The same private chat used everywhere else (Part 16).

Excel/CSV import — the complete flow

Files: components/admin/ExcelImportPanel.tsx , app/api/admin/projects/import/route.ts , lib/excelImport.ts (parsing), lib/admin.ts (writes).

- 1. **File types:** .xlsx, .xls, .csv — read with SheetJS (xlsx package). Max 10MB.

2. **Multiple sheets:** yes — every sheet is scanned; a sheet with no recognizable header row (e.g. a cover page) is silently skipped.
3. **Column detection:** fuzzy alias matching, not fixed column names — headers are normalized (lowercased, punctuation stripped) and matched against alias lists, e.g. Department also matches "dept," "trade," "category," "discipline"; Quantity also matches "qty," "target," "estimated quantity"; Unit also matches "uom," "units"; Amount also matches "value," "budget," "contract value," "cost."
4. **Preview before save:** yes, two-step. The same endpoint is called twice with the same file — once without a `commit` flag (returns a preview: sheets scanned, departments found, work-item count, row-level issues, nothing written), then again with `commit=1` (performs the actual writes).
5. **Tables written:** only departments and `work_items` (never projects/companies — the target project already exists). Departments are matched by name (case-insensitive) or created; work items are matched by department + line-item-number (or description) and either updated or inserted.
6. **Invalid rows:** skipped individually with a logged reason (e.g. "Missing department — row skipped"), never fail the whole import. A negative quantity/amount is nulled out with a warning rather than rejecting the row. The whole import only fails if literally zero rows were recognized across every sheet.
7. **After confirmation:** a success banner shows counts of departments/work items created vs. updated plus any skipped-row count; the Admin Setup page data is refreshed in place (no redirect).

NEEDS MANUAL VERIFICATION Whether the standalone `raw-inputs` storage bucket (created in an early migration) is still used anywhere — no code reference to it was found; it may be legacy.

Part 8

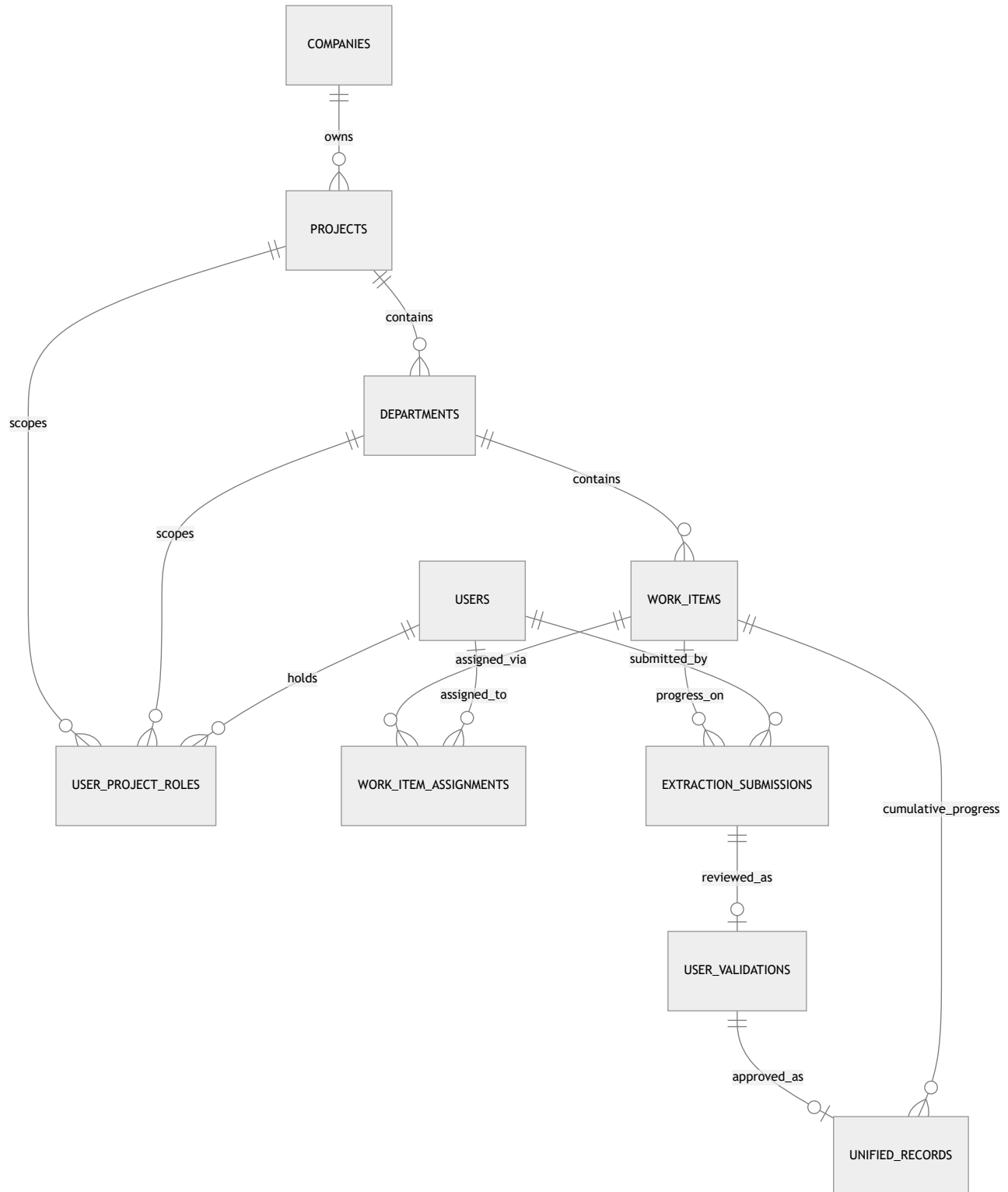
Database / Data Model

Important structural fact: not all of this app's tables are defined in `supabase/migrations/`. The 9 tables the whole workflow actually runs on — `companies`, `projects`, `departments`, `users`, `user_project_roles`, `work_items`, `extraction_submissions`, `user_validations`, `unified_records` — were created directly in the Supabase dashboard and are never defined by a `CREATE TABLE` statement in this repo. The migrations only *add columns to* or *reference* them. This is explicitly acknowledged in the migration files themselves ("the untracked core tables"). Their columns below are reconstructed from how the code actually queries them, not from DDL — flagged per table.

TABLE	DEFINED BY	PURPOSE	USED BY
companies	Untracked	Root of the hierarchy — a construction company.	lib/admin.ts
projects	Untracked	One construction project, belongs to a company.	lib/admin.ts, lib/workflow.ts
departments	Untracked (+col. mig. 11)	A trade/discipline within a project (Electrical, Concrete...).	lib/admin.ts, lib/workflow.ts
users	Untracked (+col. mig. 10)	Every person in the system; <code>user_role</code> column only distinguishes Admin.	everywhere
user_project_roles	Untracked	The real scoping table — one Active row = this user's role+project+department.	lib/authContext.ts, lib/workflow.ts
work_items	Untracked	A unit of work: planned quantity, unit, scheduled value, CSI code (plain text column — no separate CSI-divisions table exists).	lib/admin.ts, lib/workflow.ts
work_item_dependencies	Migration 4	"Work item X can't start until Y is done."	lib/workflow.ts, lib/admin.ts

TABLE	DEFINED BY	PURPOSE	USED BY
work_item_assignments	Migration 5	Which specific worker is assigned to which specific work item.	lib/workflow.ts
extraction_submissions	Untracked	Stage 1: a Worker's raw submitted update (text/photo/voice result).	lib/workflow.ts
user_validations	Untracked	Stage 2: the Subcontractor's forward/correction, later the Contractor's approve/roll-back decision.	lib/workflow.ts
unified_records	Untracked	Stage 3: the final approved ledger row — the only thing progress math reads from.	lib/workflow.ts, lib/dashboard.ts
admin_delegations (+3 child tables)	Migrations 6, 8, 9	Who was delegated what, on which project/department, for how long.	lib/delegation.ts
notifications	Migration 7, 12	In-app notifications (approvals, returns, etc.).	lib/notifications.ts
standard_departments	Migration 11	20 seeded trade names, used to prefill new departments.	lib/admin.ts
live_updates	Migration 13, 14	Field photo/voice "proof of activity" — separate from the progress pipeline entirely.	lib/liveUpdates.ts
chat_messages	Migration 17, 18	Private/team chat (Part 16).	lib/chat.ts
submissions / extracted_results	Migration 1 (dead)	An old, now-unused test-app schema — RLS access removed in migration 15, zero code references remain.	none (dead)

Relationship diagram



This corrects the guessed flow in the original outline: it is **not** a single straight chain from "Submission → Validation → Unified Record → Dashboard." A submission only reaches

`unified_records` after passing through *two* separate gates (Subcontractor forward, then Contractor approval) inside `user_validations` — see Part 13.

NEEDS MANUAL VERIFICATION Exact column types/constraints/indexes and current RLS policy state for every "untracked" table above — these were reconstructed from how the code queries them, not from a `CREATE TABLE` statement, since none exists in this repo. Also `reporting_periods` (queried once in `lib/workflow.ts`) has no migration at all — its existence/shape needs direct verification in the Supabase dashboard.

Part 9

Contractor Flow

Login → `/workflow/supervisor` → `ContractorTabs.tsx` . Seven tabs, all inside the shared `DashboardShell` .

TAB	SHOWS	DATA SOURCE	ACTIONS
Today's Progress	Today's / yesterday's submissions + the live approval queue.	<code>SupervisorPanel</code> , server-fetched per page load.	Approve, edit, comment, roll back (own department, or delegated).
MTD Summary	Month-to-date cumulative progress per work item.	Same component, MTD-filtered.	Read-only + planned-quantity edit.
Dashboard	Overall progress ring, project value, completed/remaining counts.	<code>DashboardPanel</code> (shared with <code>/workflow/dashboard</code>).	Filter by project/department/status/date — always visible, no dropdown.
Work Items	Every work item, progress bar, status badge, plus a "View Live Updates" button per row.	<code>DashboardPanel</code> .	Clicking "View Live Updates" jumps to the Live Updates tab pre-filtered to that one work item.
Department Progress	Progress rolled up per department.	<code>DashboardPanel</code> .	Read-only.
Live Updates	Today's field photos/voice notes, split from History.	<code>LiveUpdateFeed</code> , self-fetching, 4-min refresh (Part 15).	Read-only; filter chip shows/clears the Work-Items-triggered filter.
Communication	Private 1:1 chat + Team broadcast.	<code>ChatPanel</code> (Part 16).	Send/read messages.

Why the filters are there: the Dashboard/Work Items/Department Progress tabs all share one component (`DashboardPanel`) whose data is scoped server-side to whatever the Contractor is authorized to see — the filter bar (Project, Department, Status, From/To date) narrows *within* that already-authorized scope; it never widens it (confirmed: the API re-verifies scope server-side regardless of what a filter value in the URL claims).

Restricted: cannot create a project; cannot approve outside their own department without a delegation; cannot directly edit a Subcontractor's user account.

Part 10

Subcontractor Flow

Login → `/workflow/foreman` → `ForemanTabs.tsx` . A Subcontractor (FOREMAN) is not "assigned" by the Contractor in this app — **role and department are set once, by an Admin, in Admin Setup** → **Users** (Part 7). From then on, the Foreman's department is fixed via their `user_project_roles` row; they never choose or switch it.

TAB	SHOWS	NOTES
Dashboard	KPIs for their department (assigned work, completed, pending review, progress).	Read-only.
Work Item Assignments	"Department: X [View Live Updates]" header, then a picker to assign a Worker to a Work Item.	The "View Live Updates" button sits beside the department name (per the latest UI change), not below the assignment list.
Reviews	Their pending queue (Worker submissions awaiting forward) plus a read-only "Awaiting Contractor Review" list.	Forward/correct here — this is the Stage-2 gate (Part 13).
Live Updates	Today/History field photos, scoped to their own department only.	Same component Contractor uses, server-side department-scoped.
Communication	Private chat.	Same as every role.

Can edit: assignments and submission corrections/comments within their own department.
Cannot edit: planned quantity outside their own department (needs a delegation), and cannot approve anything — only Contractor/Admin can approve.

Part 11

Worker Flow

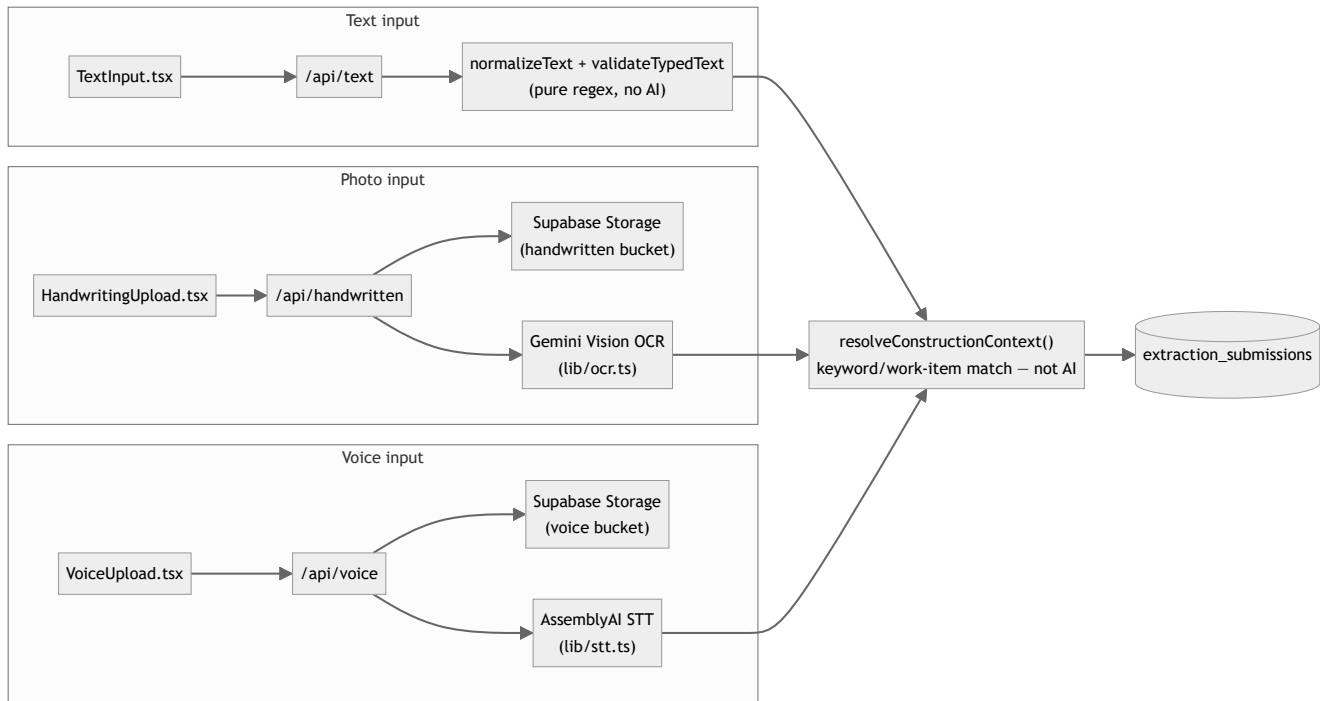
Login → `/workflow/worker` → `WorkerTabs.tsx` : Work Update, Dashboard, Work Details, History, Approved Work, Communication.

1. The Worker sees their currently assigned work item (auto-suggested if they have several, or picked from a dropdown), with Planned Quantity and Unit already shown.
2. They fill in today's update via **text, photo, or voice** (Part 12) plus an optional Live Update photo/voice note (a *separate*, evidence-only feature — Part 15).
3. On Submit, `submitWorkerProgress()` (`lib/workflow.ts`) first verifies — server-side — that this work item is actually assigned to them, then inserts one row into `extraction_submissions`.
4. That's the Worker's part done. They can see it afterward in History as "Awaiting Subcontractor Review," and later "Forwarded to Contractor — Awaiting Approval," and finally as Approved Work once it clears both gates. They cannot edit a submission after sending it — there is no Worker-facing edit function in the codebase.

Part 12

Extraction Pipeline

Three independent input paths, each ending in the same kind of database row. **Only the photo path uses an AI model at all** — text is pure parsing, and voice uses a dedicated speech-to-text service, not an LLM.



Text

`components/TextInput.tsx` → `app/api/text/route.ts`. No AI call at all — just `normalizeText()` (whitespace cleanup) and `validateTypedText()` (rejects empty/too-short input). Quantity/work-item matching is done by `lib/construction.ts`'s keyword matcher, and — in the separate Worker-submission form — `lib/progressParsing.ts` extracts a percentage (regex for "NN%") or a quantity+unit (regex built from a unit-synonym table, e.g. "cu m," "cubic meters," "cum" all match m³) from free text, explicitly refusing to guess when a sentence is ambiguous (e.g. mentions both a "planned" and a "completed" number).

Photo (handwriting/notes)

`components/HandwritingUpload.tsx` → `app/api/handwritten/route.ts`. The image is uploaded to the private handwritten Storage bucket, then sent to **Gemini** (@google/genai, model `gemini-3.6-flash` by default) with this exact prompt:

"You are extracting text from a construction site note, which may be handwritten or printed. Rules: Extract exactly what is written. Do not summarize. Do not guess unreadable words. Do not invent numbers or quantities. If a word is unclear, write [unclear]. If nothing is readable, return exactly: UNREADABLE After the extracted text, add a final line: CONFIDENCE: 0.xx"

Gemini's job here is **transcription only** — it returns raw text + a confidence score, not structured quantity/work-item/date fields. That structuring is done afterward by the same non-AI

`lib/construction.ts` keyword matcher used for text input.

Voice

`components/VoiceUpload.tsx` → `app/api/voice/route.ts`. Audio is uploaded to the private voice bucket, then sent to **AssemblyAI** (plain REST calls — no SDK): upload → request a transcript → poll until complete. The returned text then goes through the identical parsing path as text/photo.

Where it lands

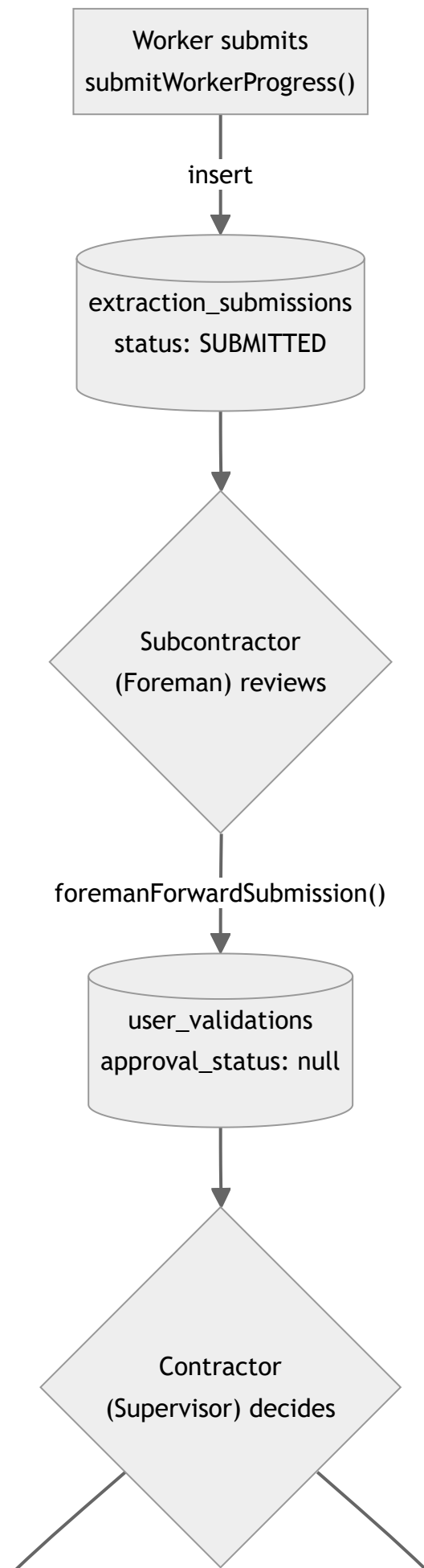
All three routes independently insert into `extraction_submissions` with `submission_status: "PENDING_REVIEW" or "REJECTED"`. **Important distinction found in the code:** these three routes are actually a separate *accuracy-testing* harness (`/dev/extraction-test`) — the real, production Worker submission flow (`WorkerSubmitForm` → `submitWorkerProgress()`) writes to the same table but with `submission_status: "SUBMITTED"`, and is what actually feeds the review/approval chain in Part 13.

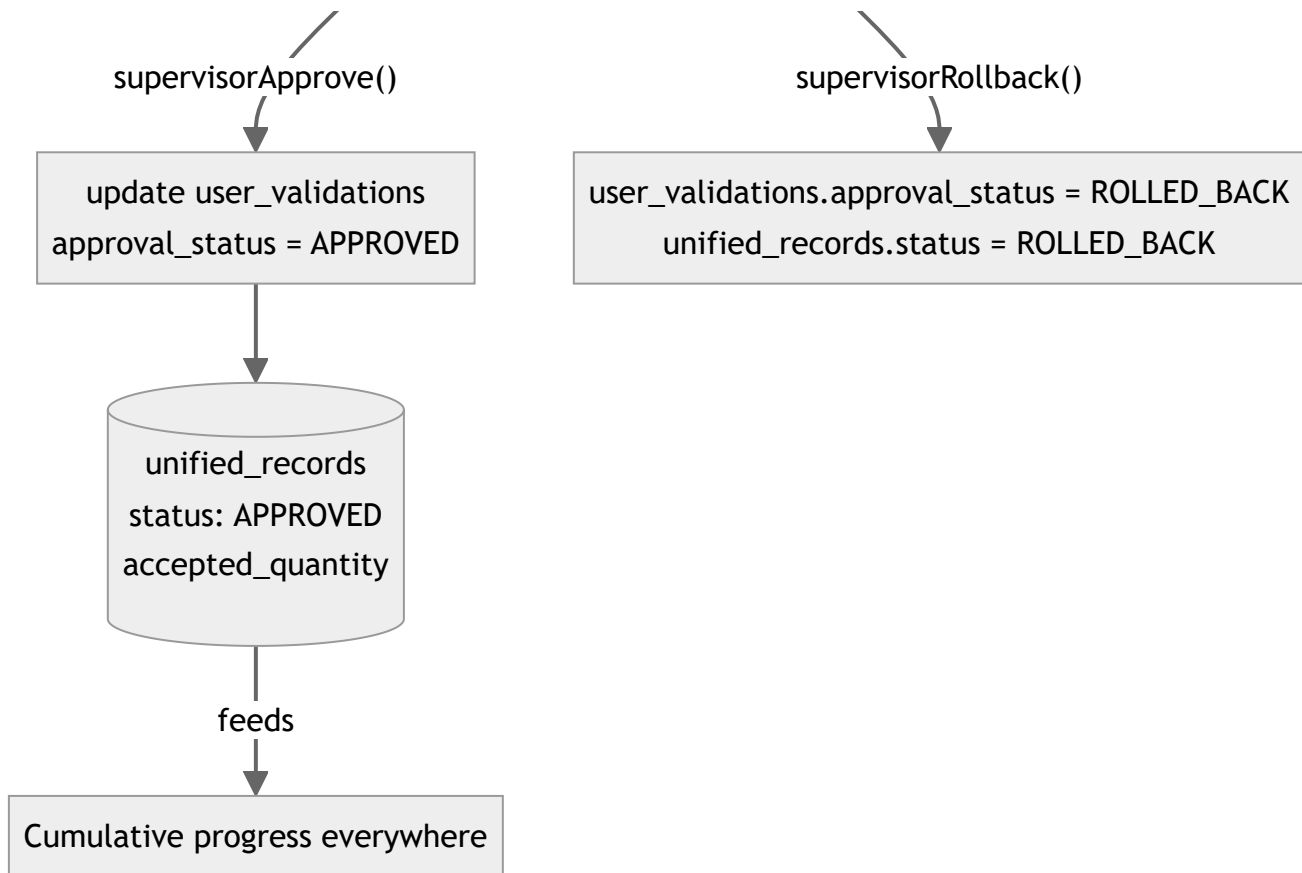
NEEDS MANUAL VERIFICATION No explicit "date" field extraction was found anywhere in the OCR/STT pipeline — dates appear to come from `created_at` timestamps, not parsed from the content itself. Also, `/dev/extraction-test` has no authentication gate in the code at all — confirm this is intentional for a production deployment.

Part 13

Review / Approval

The pipeline stage of a submission is never stored as a status field — it's *derived* from which rows exist, per the code's own module comment in `lib/workflow.ts` :





Who reviews, who approves: the Subcontractor (Foreman) *reviews/forwards* — this alone does not count toward progress. Only the Contractor (Supervisor/Manager) — or an Admin — *approves*, and that is the single action that writes to `unified_records`, the table every progress calculation reads from.

Before approval: the raw submission sits in `extraction_submissions`; once forwarded, a working copy (with any correction) sits in `user_validations` — neither counts toward the project's official numbers. **After approval:** a snapshot lands in `unified_records` — this is the only thing that counts, and a roll-back flips it back out (row kept, never deleted, for audit history).

Planned quantity

The total amount of work expected for this item (e.g. 500 m³) — set in Admin Setup / Work Items, editable later by Admin or the department's own Contractor/Subcontractor (or a delegate).

Accepted (approved) quantity

The amount from one specific approved submission (e.g. today's 40 m³) — stored per-row in `unified_records`.

Cumulative progress

The running total of every approved quantity for that work item, summed live on every read — never stored as a single running-total field.

Scheduled value

The dollar value assigned to completing this work item (e.g. \$50,000) — used to translate progress % into an estimated dollar amount earned.

Unit of measure

What the quantity is counted in (m³, linear ft, sq ft, %...) — set once per work item and used to validate/parse every submission against it.

Part 14

Progress Calculation

Core formulas live in one shared file, `lib/calculations.ts` — used by both server and client so the math is never duplicated or drifted:

```
Progress % = (Cumulative Approved Quantity / Planned Quantity) × 100 — clamped to 0–100  
Estimated Amount = Scheduled Value × (Progress % / 100)
```

`getCumulativeApprovedQuantity(workItemId)` sums `accepted_quantity` across every `unified_records` row for that work item where `status = 'APPROVED'` — a plain SQL sum, recomputed on every page load, never cached on the work item itself.

Worked example (from the actual code's formula): Planned quantity = 500 m.

- Day 1: Worker submits 100 m → Subcontractor forwards → Contractor approves → `unified_records` row #1 (100 m, APPROVED). Cumulative = 100 → Progress = **20%**.
- Day 2: another 150 m goes through the same two gates → row #2 (150 m, APPROVED). Cumulative = 100 + 150 = 250 → Progress = **50%**.

This matches the example in the original brief exactly, and is verified directly against `getWorkItemCurrentStatus()` in `lib/workflow.ts`.

If a work item has no configured planned quantity (legacy percentage-only mode), the app falls back to the latest single approved submission's own stored percentage instead of a quantity ratio.

Part 15

Live Updates

Live Updates exist to give a Contractor/Subcontractor quick visual proof of site activity ("what's happening right now") — **completely separate from the progress pipeline**. A live update photo/voice note is never approved, rejected, counted toward progress, or read by any calculation logic (confirmed: its own migration explicitly documents it as "never reused by anything that scans the extraction pipeline").

Today vs. History: `LiveUpdateFeed.tsx` fetches every Active live update the caller is scoped to see (all of them, no server-side date filter), then splits that same array client-side by local calendar day — "Today" is today's day-key, "History" is everything else, grouped by day, newest first.

Is it real-time? No — confirmed from the code, not assumed. It's fetched once on mount and then re-fetched every **4 minutes** (`REFRESH_INTERVAL_MS`), purely to re-sign the private storage URLs before their 5-minute expiry — not for near-real-time freshness. There is no WebSocket/subscription anywhere in this feature.

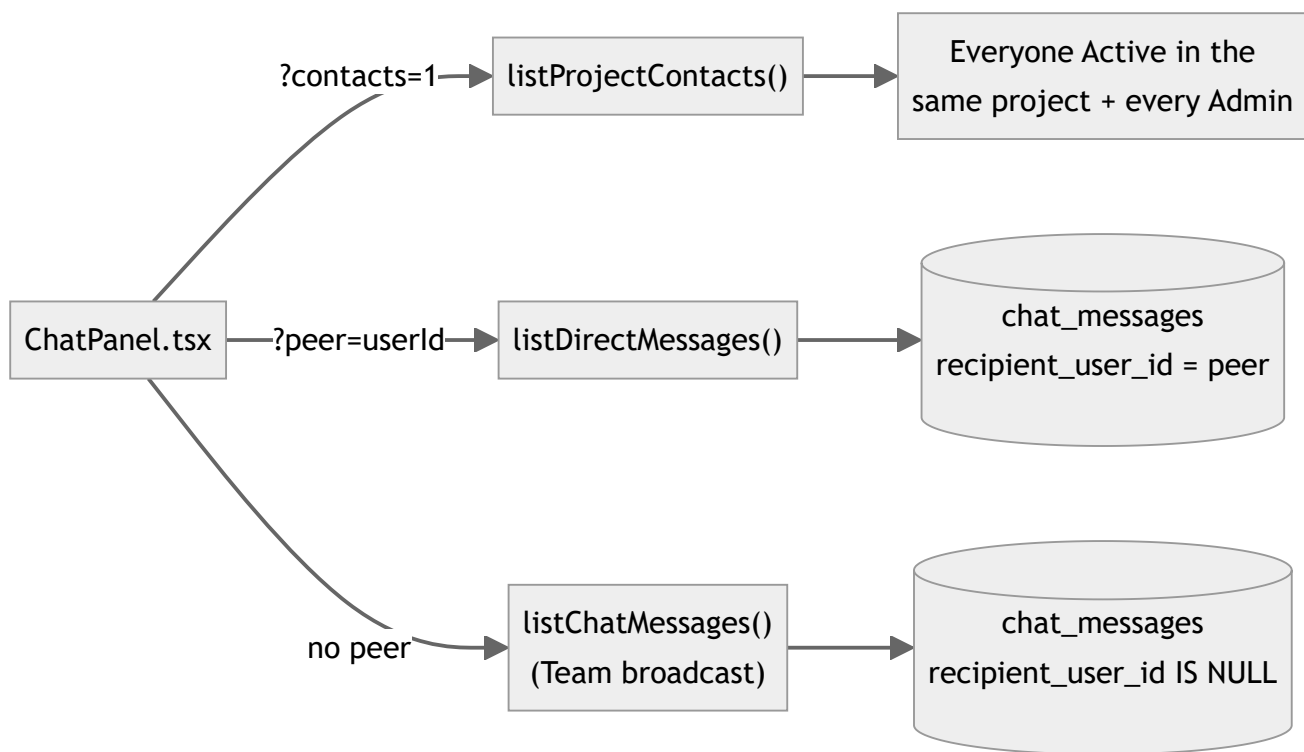
"View Live Updates" buttons: Contractor's Work Items row → sets a client-side filter (by work-item code) and switches to the Live Updates tab, which then shows only that item's updates (a "Show all" chip clears it) — filtered from the same already-fetched payload, no new API call. Subcontractor's "Department: X [View Live Updates]" button simply switches to the same tab, which is already server-scoped to their department.

Part 16

Communication

Originally a single project-wide broadcast board; migration

000000000000018_chat_direct_messages.sql added a nullable `recipient_user_id` column to the existing `chat_messages` table — additive, no data lost. A message with `recipient_user_id = null` is a broadcast (shown as the pinned "**Team**" conversation); a message with it set is a private 1:1 direct message.



The contact list is intentionally **open within a project**, not gated by a request/accept flow — a Worker can message any Subcontractor/Contractor/Admin sharing their project directly. Before a message is sent, the server re-verifies the recipient is actually one of the caller's valid contacts (never trusts a client-supplied user id). The table has no anon/authenticated Row Level Security policy at all — reachable only through the server's service-role client, with all authorization enforced in application code (`lib/chat.ts`), the same pattern used for Live Updates.

Part 17

Permissions / Security

Every check below is enforced **server-side** (mostly in `lib/workflow.ts` and `lib/delegation.ts`)
— UI buttons being hidden is a convenience, never the actual boundary.

QUESTION	ANSWER (CODE-VERIFIED)
Can a Worker edit their own submission/comments after submitting?	No — no such function exists anywhere in the codebase.
Can a Worker self-assign work?	No — submitting against an unassigned work item is explicitly rejected server-side.
Can a Subcontractor assign a Worker?	Yes — within their own department only.
Can a Contractor edit a Subcontractor's data directly?	Partially — they can act on submissions/work items in a shared department (review/approve), but cannot edit the Subcontractor's own user account (that's Admin/USER_MANAGEMENT-only).
Can a Contractor create a project?	No — Admin-only, unconditionally.
Can a <i>delegated</i> Contractor create a project?	No — explicitly excluded even from delegation; the API route calls the strict Admin-only check regardless of any PROJECT_MANAGEMENT delegation.
Can a Subcontractor edit planned quantity?	Yes, own department — same rule as Contractor: own department automatically, other departments only with a PLANNED_QUANTITY_MANAGEMENT delegation (delegation is only ever grantable to a Contractor, not a Foreman).
Can one department see another department's submissions?	No , by default — every queue/list query filters by the caller's own <code>departmentId</code> . Only an Admin (unscoped) or a Contractor holding an active cross-department delegation can see beyond their own department.

Part 18

Delegation

Admin Setup → Delegations lets a real Admin temporarily extend specific permissions to a Contractor (SUPERVISOR/MANAGER) — never to a Subcontractor or Worker — scoped to explicit project IDs (and optionally specific department IDs within them), with a start/end time window.

PERMISSION	GRANTS
WORK_ITEM_MANAGEMENT	Create/edit work items outside the Contractor's own department.
WORKER_ASSIGNMENT	Assign/remove workers on work items outside their own department.
PLANNED_QUANTITY_MANAGEMENT	Edit a work item's planned quantity/unit cross-department.
PROGRESS_REVIEW	Review/approve/roll back submissions cross-department.
PROJECT_MANAGEMENT	Edit an already-existing delegated project's name/location — never creation.
DEPARTMENT_MANAGEMENT	Edit delegated departments; create a new one only if the delegation covers the whole project.
USER_MANAGEMENT	Create/edit operational (non-Admin) users within delegated scope.

Never delegable, under any permission (hard-coded ALWAYS_ADMIN_ONLY list): Company management, creating a brand-new project, creating/promoting an Admin account, removing the original Admin, managing delegation itself, and anything touching auth/Supabase credentials/RLS.

How it's checked: `hasDelegatedPermission(userId, permission, {projectId, departmentId})` looks for an Active delegation row, currently inside its time window, that includes that permission, includes that project, and either has no department restriction (whole-project) or explicitly includes that department. `assertAdminOrDelegated()` wraps this everywhere a delegate-eligible action happens; `assertAdminRole()` is used for the handful of things that stay Admin-only no matter what.

Part 19

Storage

CONTENT	BUCKET	PUBLIC?	PATH PATTERN	READ BACK VIA
Handwriting/photo OCR uploads	handwritten	Private	<random-uuid>.<ext>	Not re-served to the UI after processing (processed once at upload time).
Voice recordings	voice	Private	<random-uuid>.<ext>	Same — processed once, not re-served.
Live Update photos/audio	live-updates	Private	<uploader-user-id>/<random-uuid>.<ext>	Fresh signed URL generated on every read, 5-minute expiry — never a stored permanent link.

`public/` only ever holds static, developer-authored assets (logo, login hero photo) — confirmed no user-uploaded content is ever written there. All real uploads go through Supabase Storage.

Part 20

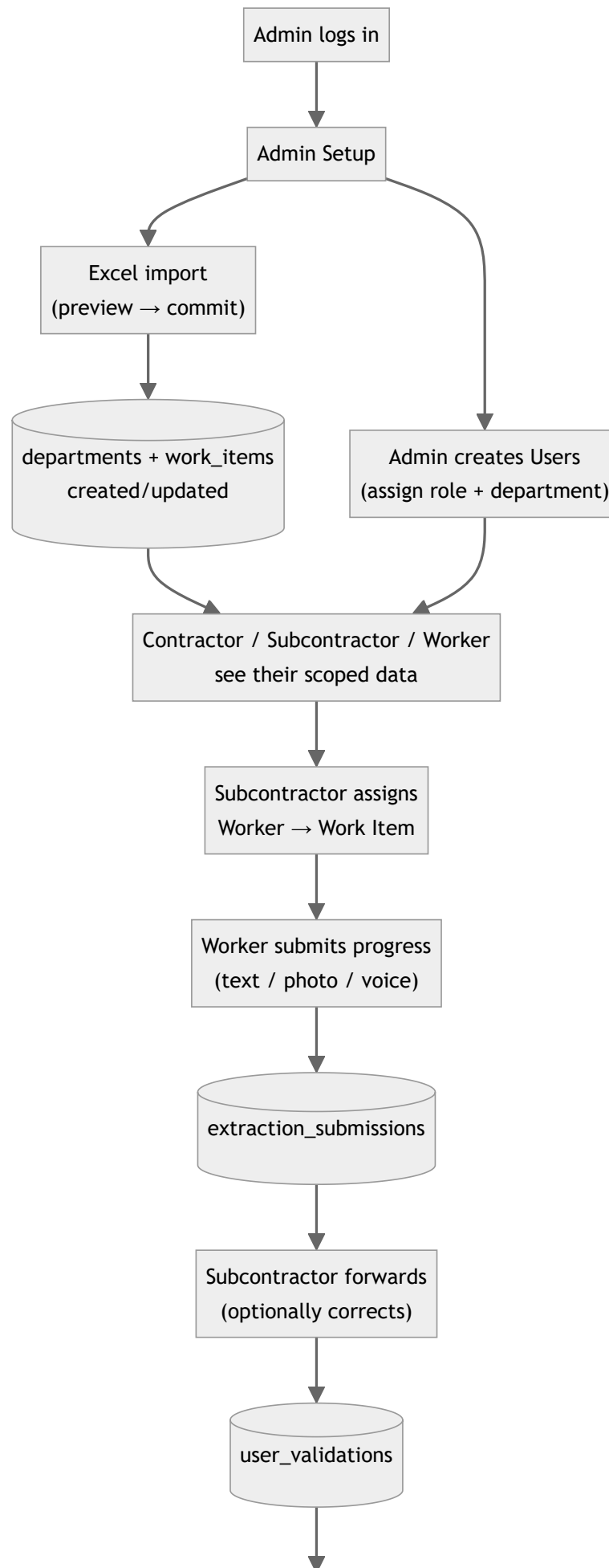
Current UI Structure

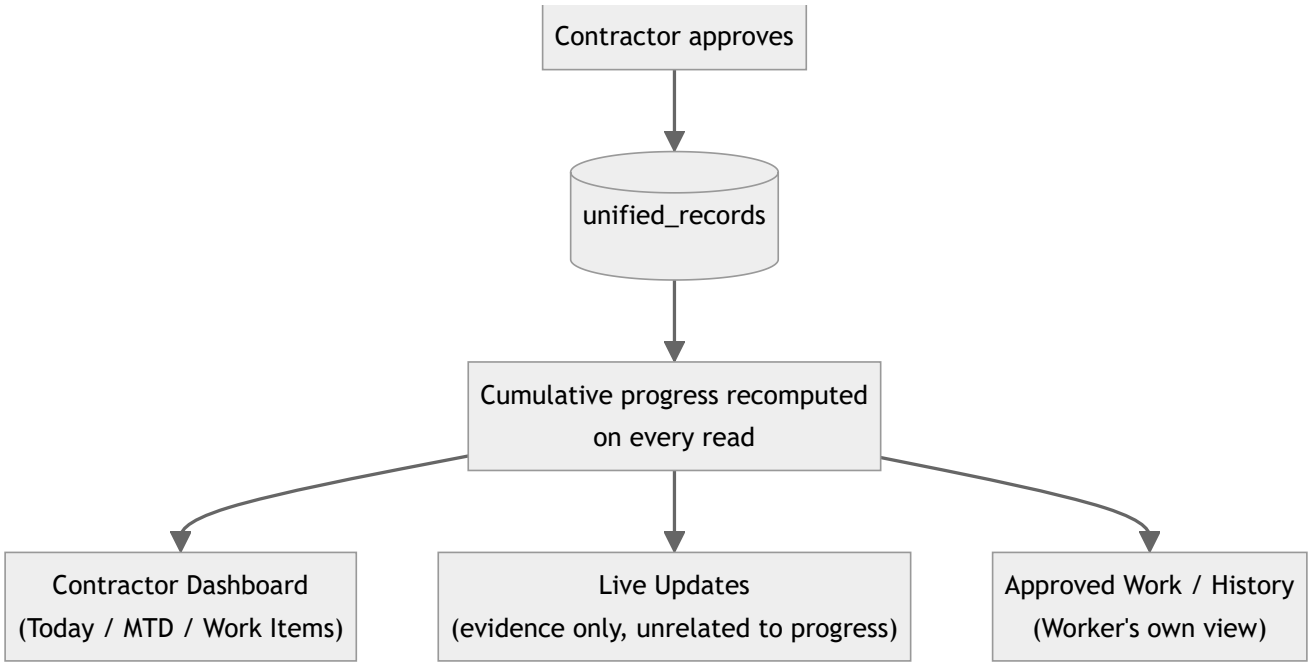
The app recently went through a layout-stability pass. **No animated background exists anywhere in the app today** — the original `BackgroundFX` particle-canvas component was deleted entirely (not just hidden), along with the nested "card floating over a differently-colored page" pattern that every dashboard page used to have. Every dashboard now fills the viewport edge-to-edge as one flat surface.

PIECE	PURPOSE TODAY
TopNav	Global top bar; "Admin Setup" and "Extraction Test (Dev)" links only render for an actual Admin (server-resolved, not just CSS-hidden).
DashboardShell / WorkerTabs	Shared sidebar+content shell (Worker duplicates the same markup rather than sharing the component) — fills the full viewport height below TopNav, no card border/shadow/rounded corners separating it from the page.
Construction Loader	A small fixed bottom-left HardHat icon badge (<code>components/ui/ConstructionLoader.tsx</code>), shown via Next.js's own <code>app/loading.tsx</code> Suspense convention during page navigation — replaces the old framework dev-only "Rendering..." indicator (which was hidden via <code>devIndicators: false</code> , since Next 16 doesn't allow reskinning it).
Communication / Live Updates	Same components reused across all four roles (Parts 15–16).

Part 22

Complete End-to-End Data Flow





Part 23

Screen-by-Screen Inventory

SCREEN	ROUTE	ROLE	PURPOSE
Login	/login	Anonymous	Credential or demo sign-in.
Role Router	/workflow	Any authenticated	Redirects to the correct role dashboard.
Worker Dashboard	/workflow/worker	WORKER	Submit progress, view history/approved work.
Subcontractor Dashboard	/workflow/foreman	FOREMAN	Assign workers, review/forward submissions.
Contractor Dashboard	/workflow/supervisor	SUPERVISOR/MANAGER	Approve progress, department/project dashboard.
Submission History	/workflow/supervisor/history	Contractor	Older submissions beyond Today/Yesterday.
Aggregate Dashboard	/workflow/dashboard	Any except Worker	Cross-department/project view.

SCREEN	ROUTE	ROLE	PURPOSE
Admin bootstrap	/admin	Anonymous (first-run only) / Admin	Create the very first Admin
Admin Setup	/admin/setup	Admin or delegate	Company/Project/Department Item/User/Delegation management
Account	/account	Any authenticated	Session/account status, sign out
Reset Password	/reset-password	Anonymous	Request a reset email.
Extraction Test	/dev/extraction-test	No gate found	OCR/voice/text accuracy-testing harness. unrelated to real world



Part 24

API / Server Action Inventory

ROUTE	METHODS	PURPOSE
/api/admin/bootstrap	POST	Create the first Admin (self-gated: zero Admins must exist).
/api/admin/companies	POST	Create a company — Admin only.
/api/admin/projects	POST, PATCH	Create (Admin only) / edit (Admin or delegate) a project.
/api/admin/projects/import	POST	Excel import — preview or commit.
/api/admin/departments	POST, PATCH	Create/edit departments.
/api/admin/work-items	POST, PATCH	Create/edit work items.
/api/admin/users	POST, DELETE	Create/remove user role assignments.
/api/admin/link-auth	POST	Link a user record to a real Supabase Auth account.
/api/admin/delegations	GET, POST, DELETE	List/create/revoke delegations.
/api/workflow/worker	GET, POST	Worker dashboard data / submit progress.
/api/workflow/foreman	GET, POST	Foreman queue / forward action.
/api/workflow/foreman/assignments	GET, POST	Worker↔work-item assignment board.
/api/workflow/supervisor	GET, POST	Contractor queue; action: approve / edit / comment / rollback.
/api/workflow/planned-quantity	PATCH	Edit a work item's planned quantity.
/api/workflow/dashboard	GET	Aggregate dashboard data (scope re-verified server-side).
/api/workflow/live-updates	GET, POST	Field photo/voice evidence posts.

ROUTE	METHODS	PURPOSE
/api/workflow/chat	GET, POST	Team + private 1:1 messaging.
/api/workflow/notifications	GET, POST	Caller's own notifications only.
/api/handwritten	POST	Photo OCR (Gemini Vision).
/api/voice	POST	Voice transcription (AssemblyAI).
/api/text	POST	Typed-text validation.
/api/user-validation	POST	Extraction-test self-validation (dev harness).

Part 25

Most Important Functions

FUNCTION	FILE	WHY IT MATTERS
<code>getCurrentUser() / requireCurrentUser()</code>	<code>lib/session.ts</code>	The one place "who is making this request" is actually verified, server-side, every time.
<code>getUserContext() / isAdminUser()</code>	<code>lib/authContext.ts</code>	The one place a caller's role + project + department are resolved.
<code>submitWorkerProgress()</code>	<code>lib/workflow.ts</code>	Stage 1 of the real progress pipeline; verifies work-item assignment first.
<code>foremanForwardSubmission()</code>	<code>lib/workflow.ts</code>	Stage 2 — Subcontractor review/forward.
<code>supervisorApprove() / supervisorRollback()</code>	<code>lib/workflow.ts</code>	Stage 3 — the only functions that write to <code>unified_records</code> .
<code>getCumulativeApprovedQuantity() / getWorkItemCurrentStatus()</code>	<code>lib/workflow.ts</code>	Where "progress %" actually gets computed, every time, from scratch.
<code>calculateProgressPercentage()</code> etc.	<code>lib/calculations.ts</code>	The single shared math used everywhere — never duplicated.
<code>assertCanManageAssignments() / assertCanReviewProgress()</code>	<code>lib/workflow.ts</code>	The actual server-side gates behind assignment and review/approval actions.
<code>hasDelegatedPermission() / assertAdminOrDelegated()</code>	<code>lib/delegation.ts</code>	Every "can a delegated Contractor do X" check funnels through here.
<code>parseProjectWorkbook()</code>	<code>lib/excelImport.ts</code>	The fuzzy-header Excel-to-structured-rows parser behind project import.
<code>geminiVisionOCR() / assemblyAISTT()</code>	<code>lib/ocr.ts / lib/stt.ts</code>	The two external AI calls in the whole app.

Part 26

Security Check

- **Authentication** is real Supabase Auth throughout, including demo login (a gated magic-link mechanism, not a bypass).
- **Authorization** is enforced in `lib/` server code on every sensitive action, independent of what the UI shows/hides.
- **Project/department isolation** is enforced by filtering every query to the caller's own `user_project_roles` scope (or an explicit delegation), not by trusting a client-supplied ID.
- **File access** for Live Updates uses short-lived (5-minute) signed URLs, never public links; handwriting/voice buckets are private and not re-exposed to the browser after processing.
- **Sensitive tables** (chat, live updates, delegation, notifications) are locked to the service-role client with no anon/authenticated RLS policy at all — the app's own server code is the only path in.

NEEDS MANUAL VERIFICATION — REAL LIMITATION FOUND, NOT INVENTED The core tables (companies, projects, users, work_items, etc.) have no RLS policy defined anywhere in this repo's migrations — their actual current RLS state can only be confirmed by checking the live Supabase project directly. `/dev/extraction-test` and its three API routes appear to have no authentication check in the code at all.

Part 27

Testing / Current Status

package.json scripts: dev, build, start, lint (ESLint 9), typecheck (`tsc --noEmit`). **No automated test framework is installed** — no Jest, Vitest, Playwright, or Cypress anywhere in the dependency tree.

What's actually been verified by tooling (run repeatedly during this project's recent UI passes): TypeScript compiles clean, ESLint passes clean, `next build` succeeds and generates all routes.

What still requires manual browser testing: every business workflow end-to-end (login → submission → approval → dashboard), the demo login flow with real credentials, and any visual/pixel-level check (no browser automation tool is available in this environment).

Part 28

Demo Flow for Tomorrow

#	SHOW	SAY	HAPPENING IN THE BACKEND
1	Login page	"One login, four roles — real Supabase authentication."	Nothing yet; page is static until submit.
2	Admin demo login	"I'll enter as Admin using the demo picker."	Magic-link session established via demoLogin.
3	Admin Setup tour	Walk Companies → Projects → Departments → Work Items → Users tabs.	Each tab's data was pre-filtered server-side before the page rendered.
4	Excel import	Upload a sample sheet, show the preview, then commit.	parseProjectWorkbook() runs twice — dry-run, then real inserts into departments/work_items.
5	Verify data	Show the newly imported work items in the list.	Straight read from the tables just written.
6	Switch to Contractor demo login	"Now as the Contractor for this project."	New session, new role/department scope resolved.
7	Switch to Subcontractor	Assign a Worker to a Work Item.	Writes work_item_assignments, gated by assertCanManageAssignments.
8	Switch to Worker	Submit a progress update by text ("completed 40 m³").	Verifies assignment, inserts extraction_submissions.
9	Submit a photo/voice update too	Show the same update via photo and voice.	Gemini OCR / AssemblyAI STT, same downstream insert.
10	Back to Subcontractor — forward it	"This is the review gate, not approval yet."	Inserts user_validations.

#	SHOW	SAY	HAPPENING IN THE BACKEND
11	Back to Contractor — approve it	"Now it becomes official progress."	Writes unified_records — the number that counts.
12	Contractor Dashboard	Show the progress ring/percentage update live.	Recomputed on read from unified_records.
13	Live Updates	Show the field photo, Today vs History split.	Independent of the above — evidence only.
14	Communication	Send a private message Worker → Subcontractor.	Only those two see it — chat_messages.recipient_user_id set.

Part 29

Beginner Glossary

Frontend

The part that runs in the browser — what the user actually sees and clicks.

Backend

The part that runs on a server — checks permissions, talks to the database, never trusts the browser.

Database

Where all the real data permanently lives — projects, users, submissions, approvals.

Supabase

A hosted PostgreSQL database + authentication + file storage service — this app's entire backend infrastructure.

Next.js

The framework that connects frontend and backend in one project — pages, API routes, and server logic all live together.

React

The library Next.js uses to build the actual screen/component tree.

TypeScript

JavaScript with types — catches a whole class of bugs (wrong data shape) before the app ever runs.

API

A URL the frontend can call to ask the backend to do something (e.g. "submit this progress update").

Server Action

A backend function called directly from a form, without hand-writing an API route for it (used for login).

Authentication

Proving who you are (email + password).

Authorization

Deciding what you're allowed to do, once we know who you are.

Migration

A saved, ordered SQL script that changes the database structure — a history of every schema change.

RLS (Row Level Security)

A PostgreSQL feature that can restrict which rows a given request is allowed to see — several tables here rely on it being off for anon access and locked to server-only code instead.

Gemini

Google's AI model — used here only to read handwritten/printed text out of a photo.

AssemblyAI

A speech-to-text service — turns a voice recording into text.

Extraction submission

A Worker's raw, not-yet-reviewed progress update.

Unified record

A final, Contractor-approved progress entry — the only thing that counts toward the project's real numbers.

Cumulative progress

The running total of every approved amount for a work item, added up live every time it's shown.

Delegation

An Admin temporarily lending a specific permission to a Contractor for a specific project/department.

How it all connects

Browser → Next.js page/API → lib/ business logic + permission check → Supabase → PostgreSQL/Storage — every single request, every time.

Part 30

Final Master Summary

A. Business, in one paragraph

This app replaces paper daily-logs and manual spreadsheet re-entry with a digital chain: a Worker reports what they finished (by typing, photographing, or speaking it), a Subcontractor checks and forwards it, a Contractor approves it, and only then does it count toward the project's official progress — which every dashboard recalculates live, so nobody is ever looking at a stale, manually-updated number.

B. Technical, in one paragraph

A single Next.js 16 / React 19 / TypeScript application talks to one Supabase project for authentication, PostgreSQL storage, and file storage; all business logic and permission checks live in `lib/`, never in the browser; two external AI services (Gemini for photo OCR, AssemblyAI for speech-to-text) are called only from server code; and the whole progress-approval chain is three tables deep (`extraction_submissions` → `user_validations` → `unified_records`), with only the last one ever counted.

K. Most important files to know

`lib/workflow.ts` (the whole submission/review/approval engine), `lib/session.ts` + `lib/authContext.ts` (who + what role), `lib/delegation.ts` (temporary permission grants), `lib/calculations.ts` (the progress formula), `proxy.ts` (the front gate).

M–O. Status

✅ Complete & verified in code

Auth (real + demo), role routing, Excel import (preview+commit), the 3-stage approval chain, progress math, delegation, private chat, Live Updates, layout/UI stability pass.

⚠️ Partially implemented / limited

No automated tests exist; several core tables have no RLS policy tracked in this repo; `/dev/extraction-test` appears to have no auth gate; Admin↔Contractor "request to chat" flow was intentionally simplified to an open contact list.

NEEDS MANUAL VERIFICATION (FULL LIST)

- Exact live-database schema for every "untracked" table (no CREATE TABLE exists in this repo for 9 core tables).
- Current RLS policy state for those same core tables.
- `reporting_periods` table's existence/shape and whether it's actually in active use.
- Whether `/dev/extraction-test` and its 3 API routes are intentionally left without an auth check.
- Whether the `raw-inputs` storage bucket is still used anywhere.

Cross-check: business flow ↔ code — one confirmed mismatch worth flagging to your team

The originally-assumed flow implied a single straight line (Submission → Validation → Unified Record → Dashboard). The actual code has **two distinct human gates** before anything counts: a Subcontractor's *forward* (writes only to `user_validations`) and a separate Contractor *approval* (writes to `unified_records`). Everything else in the originally-assumed flow — Admin → Setup → Import → Project → Department → Work Items → Contractor/Subcontractor/Worker → Submission → Extraction → Review → Approval → Dashboard → Live Updates/History/Approved Work/Communication — matches the real implementation.

Compiled entirely from the repository's own source code, migrations, and configuration — no assumptions presented as fact. Items explicitly marked "Needs manual verification" were not confirmable from the code alone.